C. Enrique Israel Torres Mendoza

Investigación de arquitecturas para la seguridad y privacidad de datos.

Un **catálogo de datos** utiliza metadatos, que describen información digital, para crear un inventario informativo y de búsqueda de todos los activos de datos, pueden incluir:

- Datos estructurados (tabulares)
- Datos no estructurados, incluidos documentos, páginas web, e-mail, contenido de redes sociales, datos móviles, imágenes, audio y video
- Informes y resultados de consultas
- Visualizaciones de datos y paneles de control
- Modelos de machine learning
- Conexiones entre bases de datos

Este inventario permite a los (analistas de datos, científicos de datos, administradores de datos y otros profesionales de datos con autenticación y autorización) buscar a través de todos los activos de datos disponibles en un repositorio.

El catálogo también proporciona herramientas que permiten hacer lo siguiente:

- Buscar en el catálogo
- Automatizar el descubrimiento de datos potencialmente relevantes que no buscaron específicamente
- Gestionar el uso de los datos de conformidad con las regulaciones normativas internas y externas

Apache Atlas.

Apache Atlas

Proporciona capacidades de gestión y gobierno de metadatos abiertos para crear un catálogo de activos de datos, clasificados que permite capacidades de colaboración para analistas, científicos, gobernanza, interoperabilidad, privacidad y calidad de datos.

Funciones

- Tipos e instancias de metadatos
- Clasificación
- Linaje
- Búsqueda/Descubrimiento
- Seguridad y enmascaramiento de datos

- Interfaz de usuario para buscar entidades por tipo, clasificación, valor de atributo o texto libre
- **API REST** permite buscar por criterios complejos
- Lenguaje de consulta similar a **SQL** para entidades de búsqueda - Domain Specific Language **(DSL)**

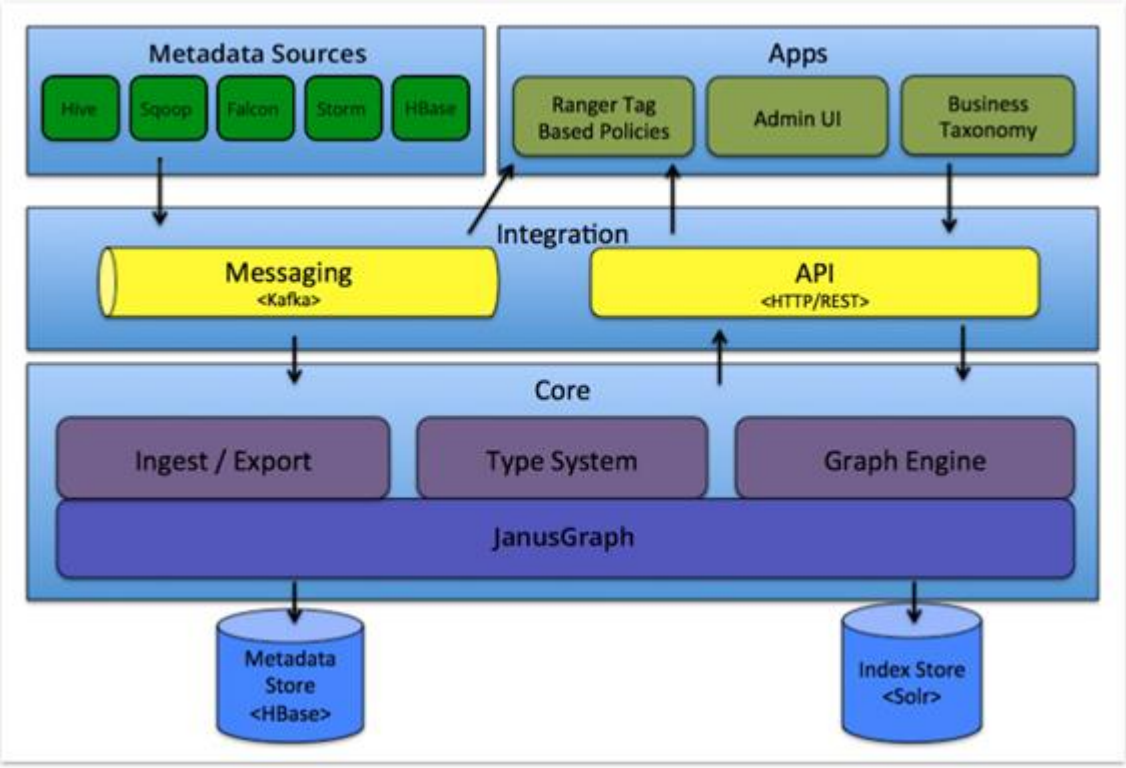
https://github.com/apache/atlas

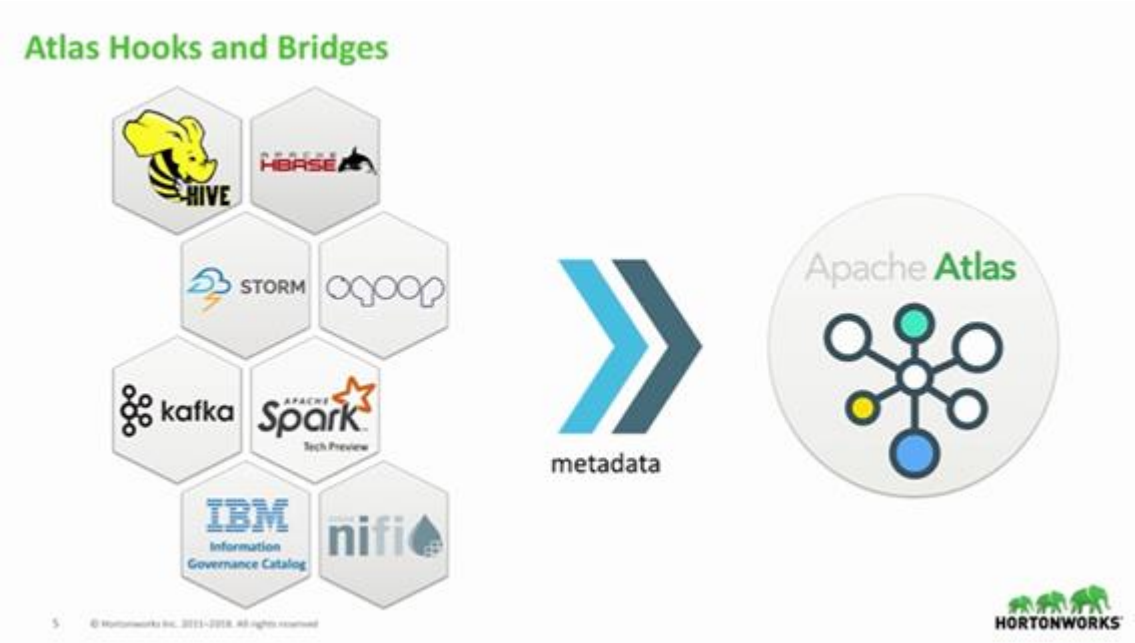
Apache Atlas es una herramienta open-source, con licencia Apache 2.0, para la gobernanza del dato la cual permite la integración con todo el ecosistema de datos de las empresas.

Atlas permite crear un almacén de metadatos centralizado para saber dónde encontrar un conjunto de datos dentro de la empresa (Data Discovery), permite saber qué cambios ha sufrido y que transformaciones se le han realizado a los datos originales a lo largo del tiempo (Data Lineage) y centraliza la seguridad para saber quién puede acceder a esos datos y quién puede modificarlos (Data Security) apoyándose en Apache Ranger.

La arquitectura de Apache Atlas y las herramientas en las que se apoya son las siguientes. Como fuente de metadatos puede utilizar: HBase, Hive, Sqoop, Storm y Kafka.

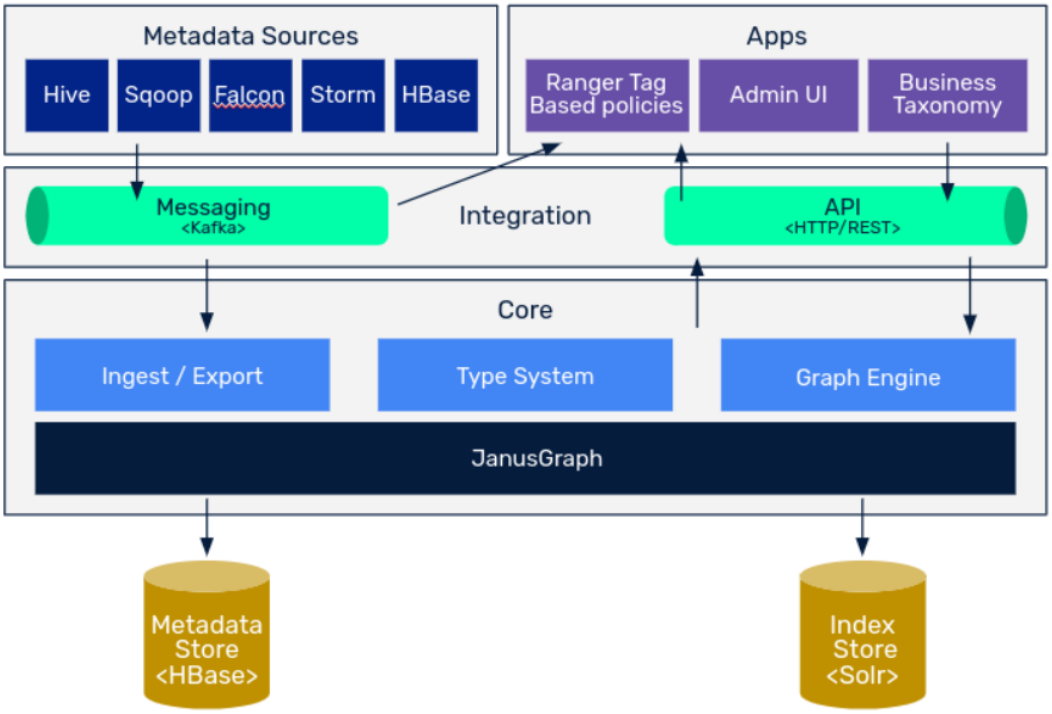
Apache Atlas permite hacer un **seguimiento de las transformaciones** que han sufrido los datos originales y en qué datos derivados han terminado para poder tener un control total sobre ellos.





Apache Atlas es una herramienta enfocada a resolver la mayoría de los principales retos del Data Governance. Esta herramienta nos permite construir un catálogo con todos los datos de nuestra organización y presentar dicho catálogo de una manera fácil e intuitiva.

Arquitectura de Apache Atlas



Core

El módulo **Type System** es el módulo principal de la herramienta, aporta la suficiente flexibilidad como para poder diseñar y almacenar cualquier tipo de información.

Para ello, utiliza los conceptos de Type y Entity de la misma manera que en programación orientada a objetos se utiliza clase e instancia. De esta forma, Apache Atlas nos provee de una herramienta que, modelando Types, podremos almacenar unidades de información respecto a esos Types llamadas Entities.

Dentro del core podemos encontrar el sistema de **ingesta y exportación** de Apache Atlas, que nos permite proveer y extraer datos del sistema. Las formas qué tenemos para hacerlo son las siguientes:

- Import: Zip, Api Rest y Kafka.
- Export: Api Rest, Incremental y Kafka (principalmente para auditoría).

Por último, el módulo **Graph Engine** se encarga de toda la parte de búsqueda e indexado de la información, para lo que utiliza el motor de grafos JanusGraph por debajo. Janus es una base de datos de grafos de código abierto qué es utilizada por Apache Atlas para representar las relaciones entre entidades dentro del catálogo. A su vez, se utiliza para realizar búsquedas sobre estas relaciones. Por ejemplo, para

encontrar todas las entidades que son derivadas de una fuente de información, estaríamos utilizando JanusGraph.

Como **backend,** Atlas hace uso de dos tecnologías de almacenamiento. La primera sería Hbase, la cual es una base de datos orientada a columnas y de tipo clave valor. Básicamente en Hbase se estaría almacenando toda la información de las entidades, toda la metainformación de nuestro catálogo.

Por otro lado, Atlas también hace uso Apache Solr, una base de datos orientada a la indexación para poder acceder a la información de una manera más rápida. Esta funcionalidad nos permitirá realizar acciones de discovering durante el uso cotidiano de la información, con las capacidades que posteriormente veremos de la herramienta.

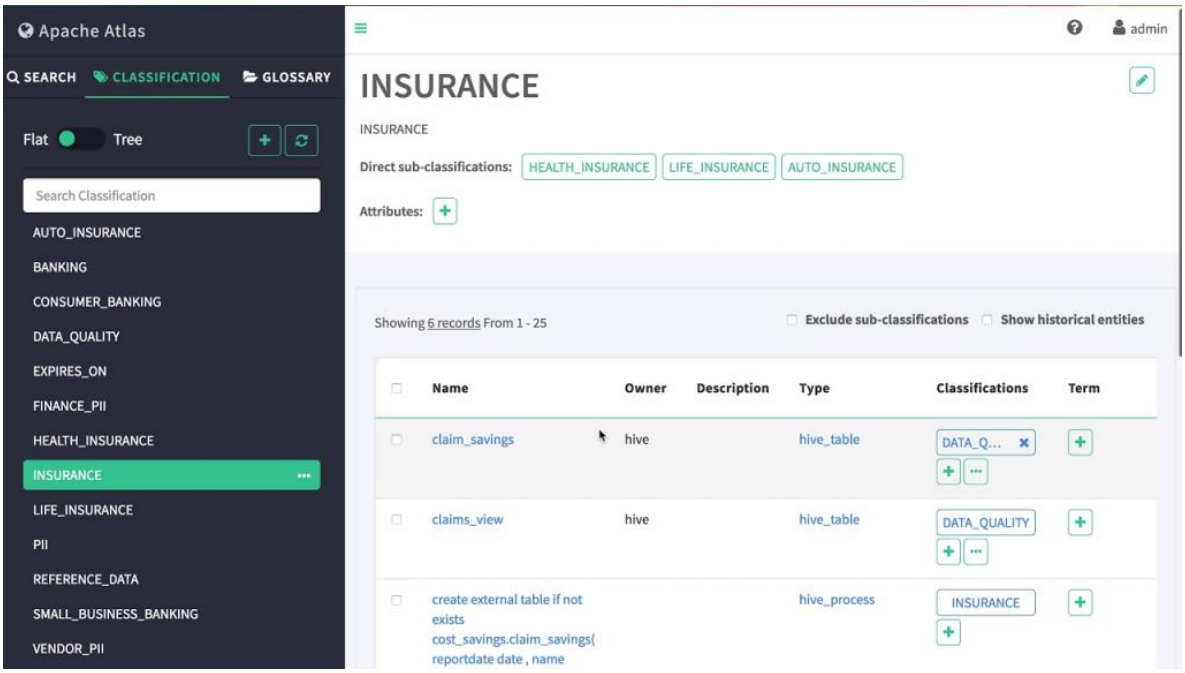
Integración

La forma principal de integración con la herramienta es a través del módulo **API**. El módulo API no es más qué una API HTTP REST que es desplegada por defecto en el puerto 21000, actualmente se encuentra la versión V2. Aparte de aportar toda la funcionalidad CRUD sobre los Types y Entities, la API también provee endpoints de consultas básicas y avanzadas y exploración.

Atlas también ofrece la posibilidad de integrar el catálogo con importaciones y exportaciones en tiempo real. La forma en la que nos ofrece esta funcionalidad es **Kafka.** Esta forma de integración es utilizada principalmente para la notificación de cambios en tiempo real e integración con otras herramientas de Data Governance como Ranger para la gestión de permisos de lectura del data lake.

Apps

La principal aplicación que se conecta con el resto de módulos es la **aplicación web,** que es desplegada por herramienta. Esta aplicación es la principal interfaz con el usuario, ya que de una forma muy usable podemos interactuar con el catálogo de metadatos.



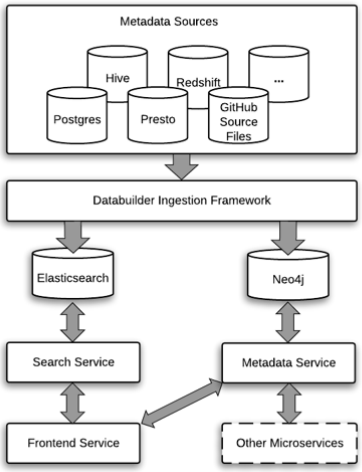
Metadata Sources

Los metadatos son aquellas fuentes que ya están soportadas de manera nativa sobre Apache Atlas.

Atlas soporta la ingesta y gestión de metadatos desde las siguientes fuentes: Hive, HBase, Sqoop, Storm y Kafka.

Amundsen.

Amundsen



Amundsen es un motor de descubrimiento de datos y metadatos indexando recursos de datos (tablas, paneles, flujos, entre otros) y potenciando una búsqueda de estilo de rango de página basada en patrones de uso (por ejemplo, las tablas altamente consultadas aparecen antes que las tablas menos consultadas).

amundsenfrontendlibrary: Servicio frontend que es una aplicación Flask con un frontend React.
amundsensearchlibrary: El servicio de búsqueda, que aprovecha Elasticsearch para las capacidades de búsqueda, se utiliza para potenciar la búsqueda de metadatos frontend.
amundsenmetadatalibrary: servicio de metadatos, que aprovecha Neo4j o Apache Atlas como capa persistente, para proporcionar varios metadatos.
amundsendatabuilder: Biblioteca de ingesta de datos para crear gráficos de metadatos e índices de búsqueda. Los usuarios podrían cargar los datos conun script de Pythoncon la biblioteca o con unDAG Airflowimportando la biblioteca.
amundsencommon: La biblioteca de Amundsen Common contiene códigos comunes entre los microservicios de Amundsen.
amundsengremlin: La biblioteca Amundsen Gremlin contiene código utilizado para convertir objetos modelo en vértices y aristas en gremlin. Se utiliza para cargar datos en un backend de AWS Neptune.
amundsenrds: Amundsenrds contiene modelos ORM para admitir bases de datos relacionales como almacén backend de metadatos en Amundsen. El esquema en los modelos ORM sigue la lógica de los modelos de generador de datos. Amundsenrds se utilizará en databuilder y metadatalibrary para el almacenamiento y recuperación de metadatos con bases de datos relacionales.

https://www.amundsen.io/

Una de las herramientas de gestión de metadatos de código abierto más comunes es Amundsen. Al igual que Facebook, no se hizo público de inmediato. Fue construido en 2018 por el equipo de ingeniería de Lyft. Amundsen estuvo disponible para el público como una herramienta de gestión de código abierto en 2019.

Existe un fuerte sentido de comunidad entre los colaboradores y usuarios de Amundsen. Muchas organizaciones han aprovechado esta herramienta de catálogo de datos de código abierto y se han basado en ella para avanzar en los esfuerzos de democratización de datos, gobernanza y servicio de metadatos.

Características clave de las herramientas de gestión de metadatos de código abierto de Amundsen.

Las siguientes son las características clave de las herramientas de gestión de metadatos de código abierto de Amundsen;

Sistemas de generación y curación de metadatos

Con la herramienta de gestión de datos de código abierto de Amundsen, a los usuarios se les presenta una descripción seleccionada manualmente al hacer clic en cualquier recurso de datos. Amundsen genera estas descripciones automáticamente.

Descubra datos fiables

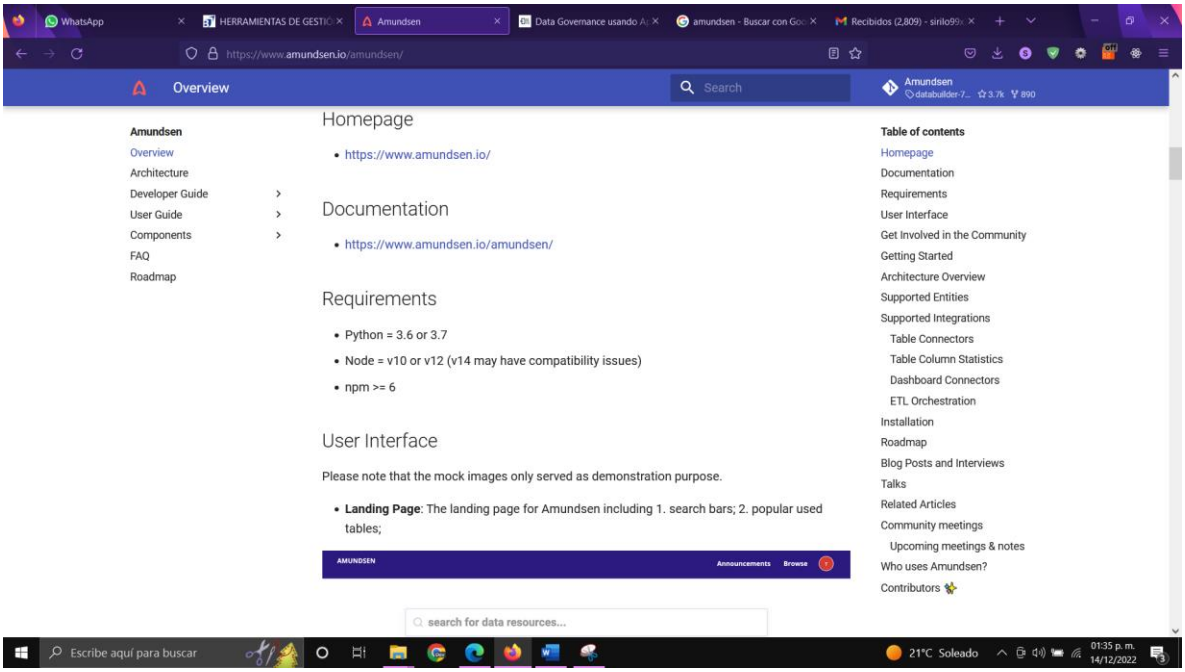
Con una sola búsqueda, Amundsen facilita el descubrimiento de datos de múltiples fuentes. Los metadatos se muestran directamente en los resultados de búsqueda.

Relacione fácilmente el contexto con los compañeros de trabajo

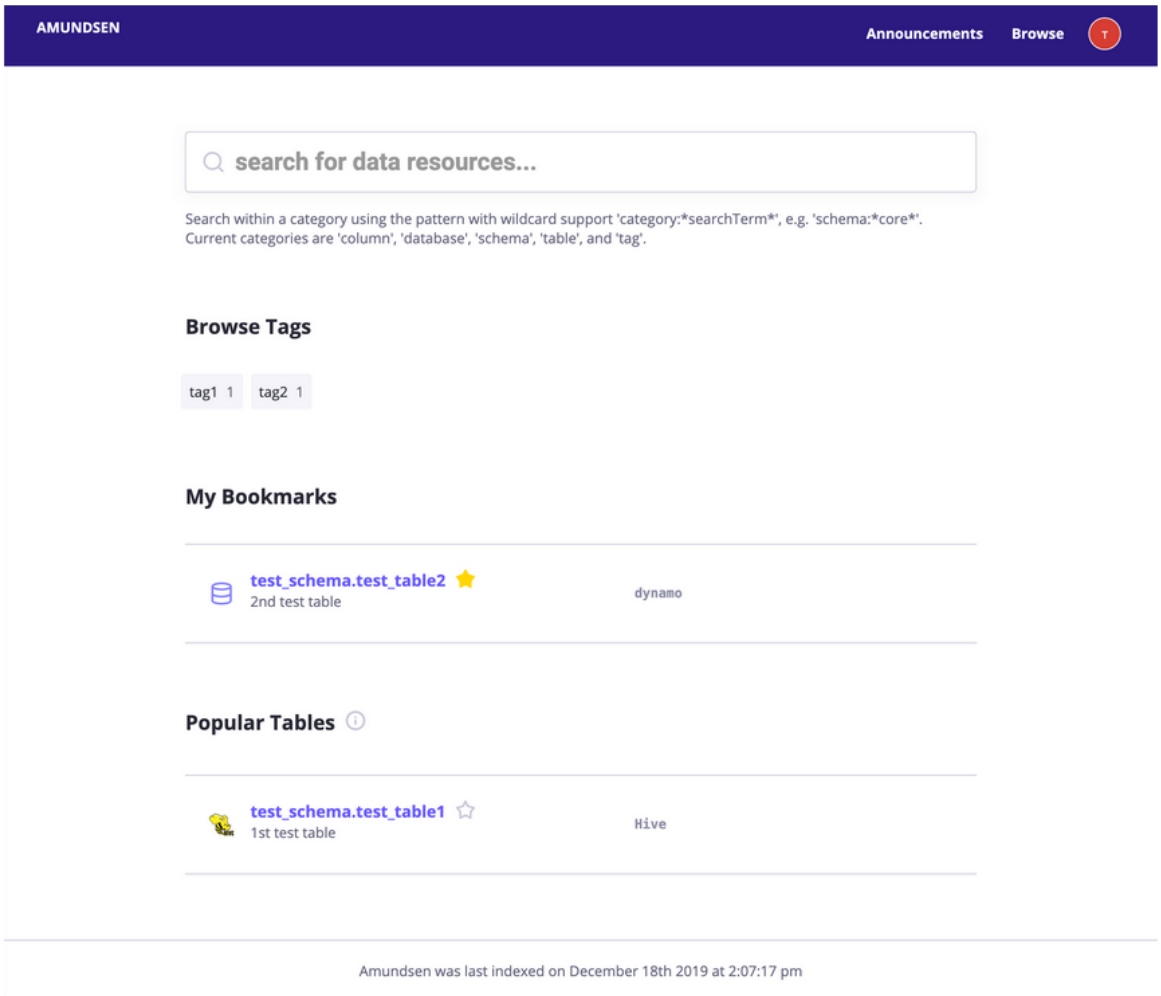
Con Amundsen, los compañeros de trabajo pueden evaluar fácilmente el contexto o los datos. Además, una única actualización o modificación se reparte por todo el sistema.

Obtener información a través del análisis de datos

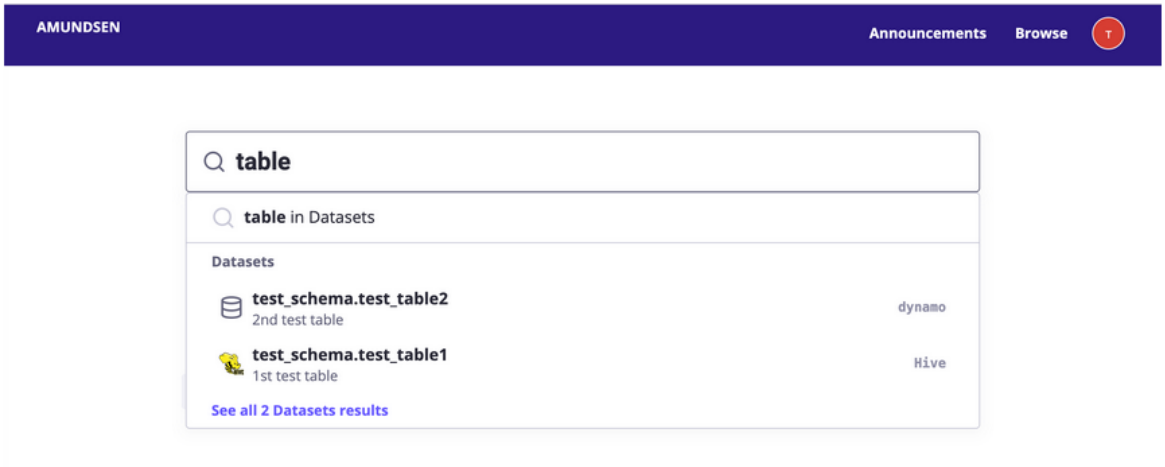
Amundsen realiza un seguimiento de sus datos. Desde los datos más utilizados hasta búsquedas y consultas, toda esta información se puede utilizar para realizar ajustes o modificar datos para un mejor uso.



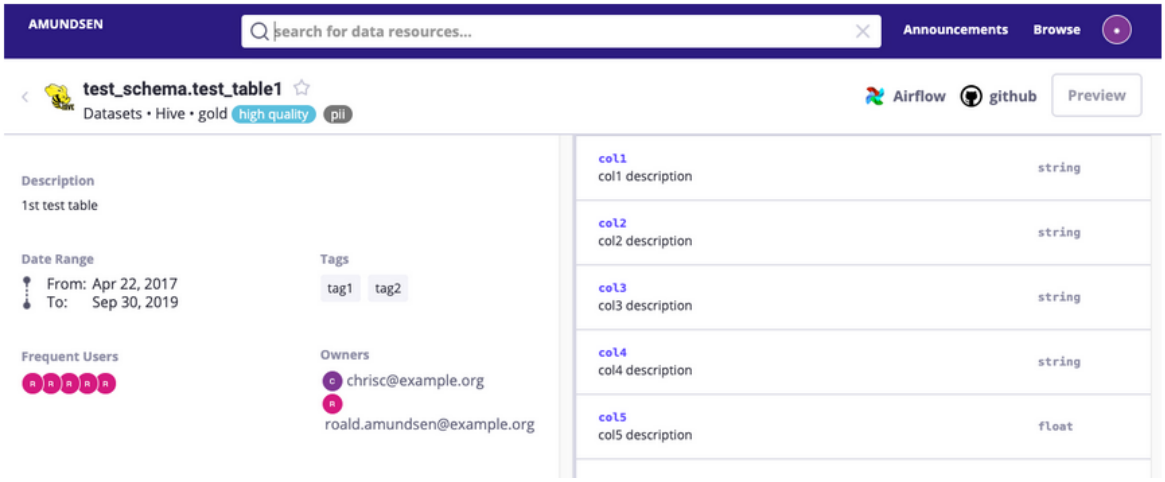
- **Landing Page**: The landing page for Amundsen including 1. search bars; 2. popular used tables;



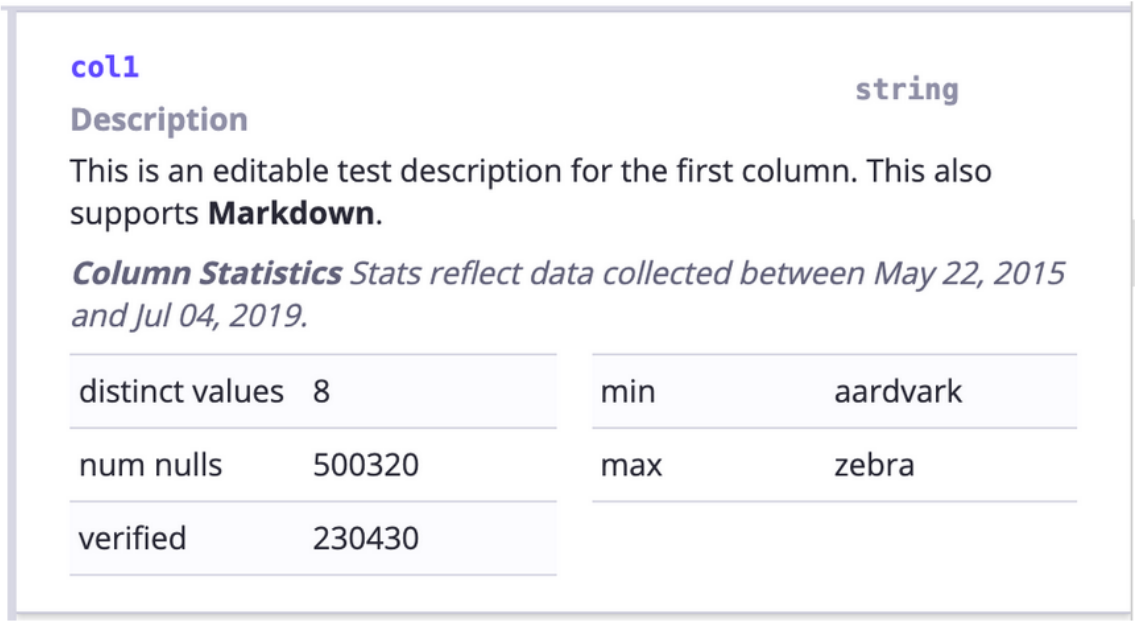
- **Search Preview**: See inline search results as you type



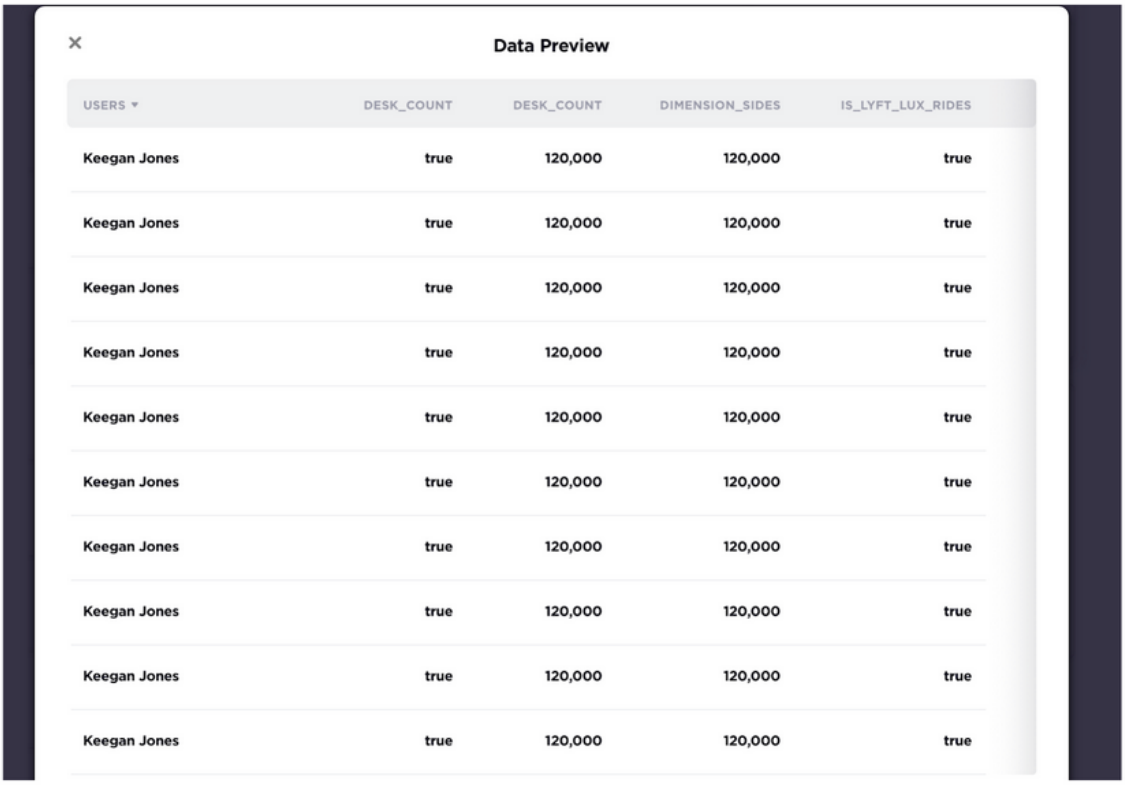
- **Table Detail Page**: Visualization of a Hive / Redshift table



- **Column detail**: Visualization of columns of a Hive / Redshift table which includes an optional stats display



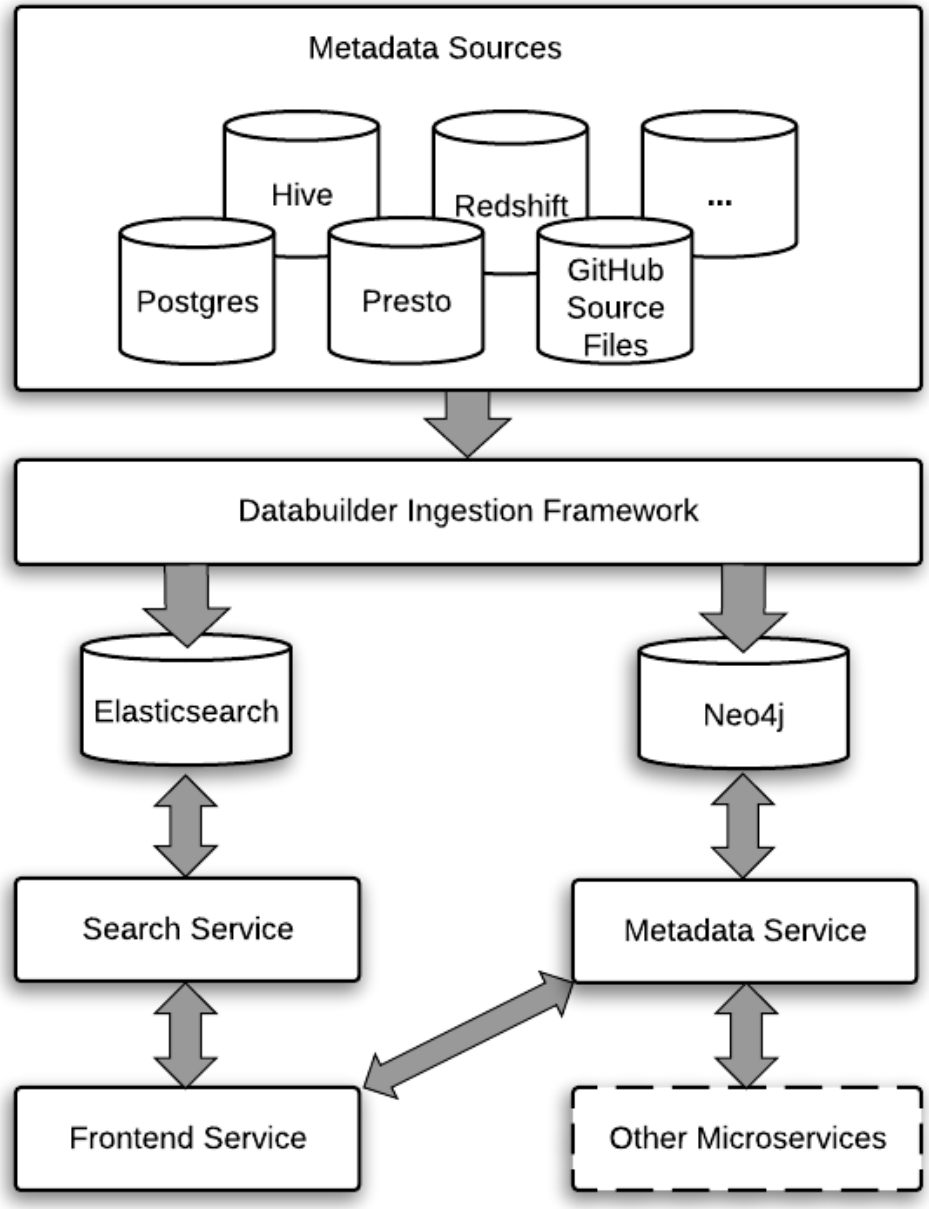
- **Data Preview Page**: Visualization of table data preview which could integrate with Apache Superset or other Data Visualization Tools.



Data Preview

USERS	DESK_COUNT	DESK_COUNT	DIMENSION_SIDES	IS_LYFT_LUX_RIDES
Keegan Jones	true	120,000	120,000	true
Keegan Jones	true	120,000	120,000	true
Keegan Jones	true	120,000	120,000	true
Keegan Jones	true	120,000	120,000	true
Keegan Jones	true	120,000	120,000	true
Keegan Jones	true	120,000	120,000	true
Keegan Jones	true	120,000	120,000	true
Keegan Jones	true	120,000	120,000	true
Keegan Jones	true	120,000	120,000	true
Keegan Jones	true	120,000	120,000	true

Arquitectura amundsen



Frontend Service.

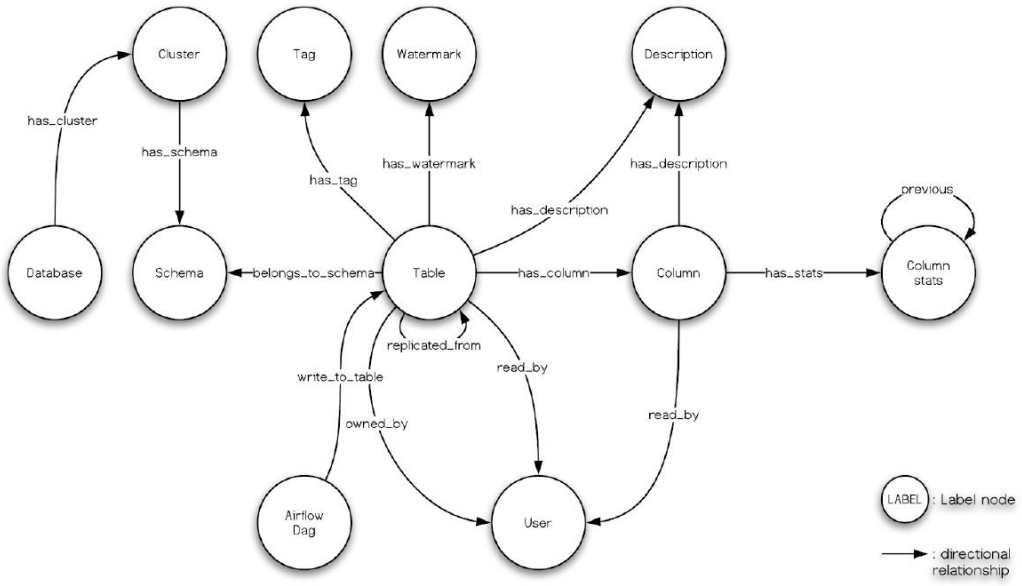
El servicio frontend sirve como portal de interfaz de usuario web para la interacción de los usuarios. Es una aplicación web basada en Flask cuya capa de representación se construye con React with Redux, Bootstrap, Webpack y Babel.

Search.

El proxy del servicio de búsqueda aprovecha la funcionalidad de búsqueda de Elasticsearch (o la API de búsqueda de Apache Atlas, si ese es el backend que eligió) y proporciona una API RESTful para atender solicitudes de búsqueda desde el servicio frontend. Esta API está documentada y se puede explorar en vivo a través de OpenAPI, también conocido como "Swagger". Actualmente, solo los recursos de tabla están indexados y se pueden buscar. El índice de búsqueda se crea con el publicador de búsqueda elástica de databuilder.

Metadata.

El servicio de metadatos actualmente utiliza un proxy Neo4j para interactuar con la base de datos gráfica de Neo4j y sirve los metadatos del servicio frontend. Los metadatos se representan como un modelo gráfico:



El diagrama anterior muestra cómo se modelan los metadatos en Amundsen.

Databuilder.

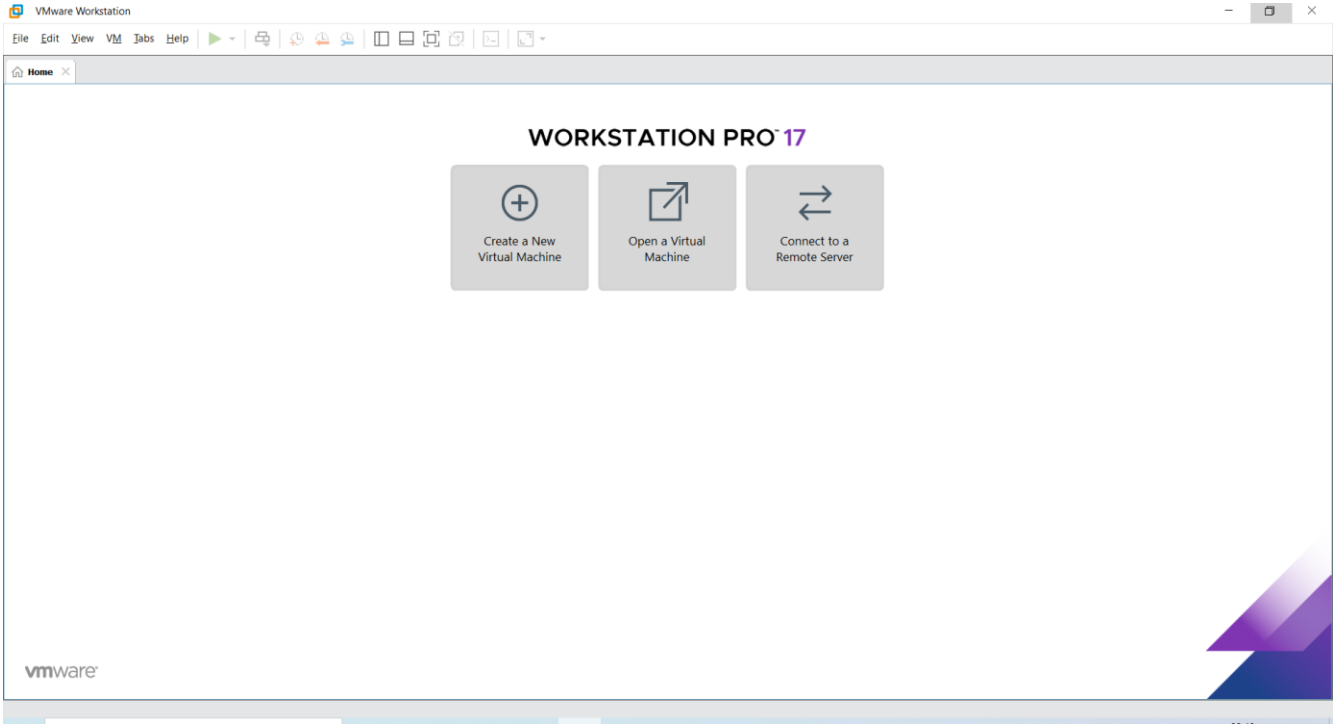
Amundsen proporciona una biblioteca de ingesta de datos para construir los metadatos.

Además del "uso real", el generador de datos también se emplea como una herramienta útil para ingerir algunos datos de demostración "precocinados" utilizados en la guía de inicio rápido. Esto le permite tener una muestra de datos superpequeña para explorar, por lo que muchas de las funciones de Amundsen están iluminadas sin que tenga que configurar ninguna conexión a bases de datos, etc. para ingerir datos reales.

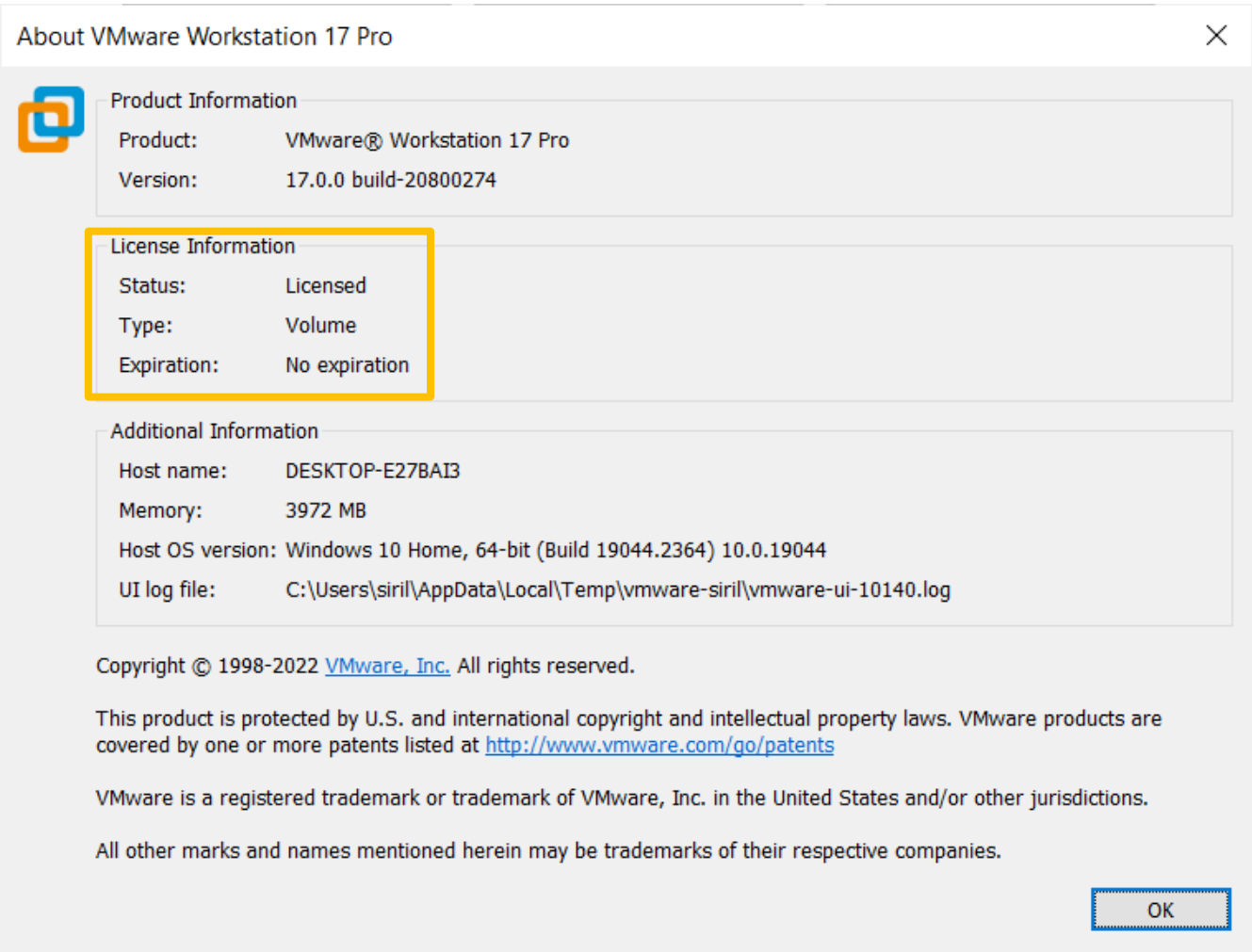
Instalación de sistema operativo Linux Red Hat.

Para trabajar los contenedores en Podman, es necesario y recomendable contar con un sistema operativo Linux, en este caso Red Hat, ya que este sistema ya tiene integrado por defecto todos los comandos de Podman, con lo cual podemos trabajar con contenedores sin tener que instalarlo manualmente en un sistema distinto.

Para instalar Red Hat en este caso se utilizó una máquina virtual mediante MVWARE WORKSTATION, se tuvo que instalar este programa para poder crear la máquina virtual.



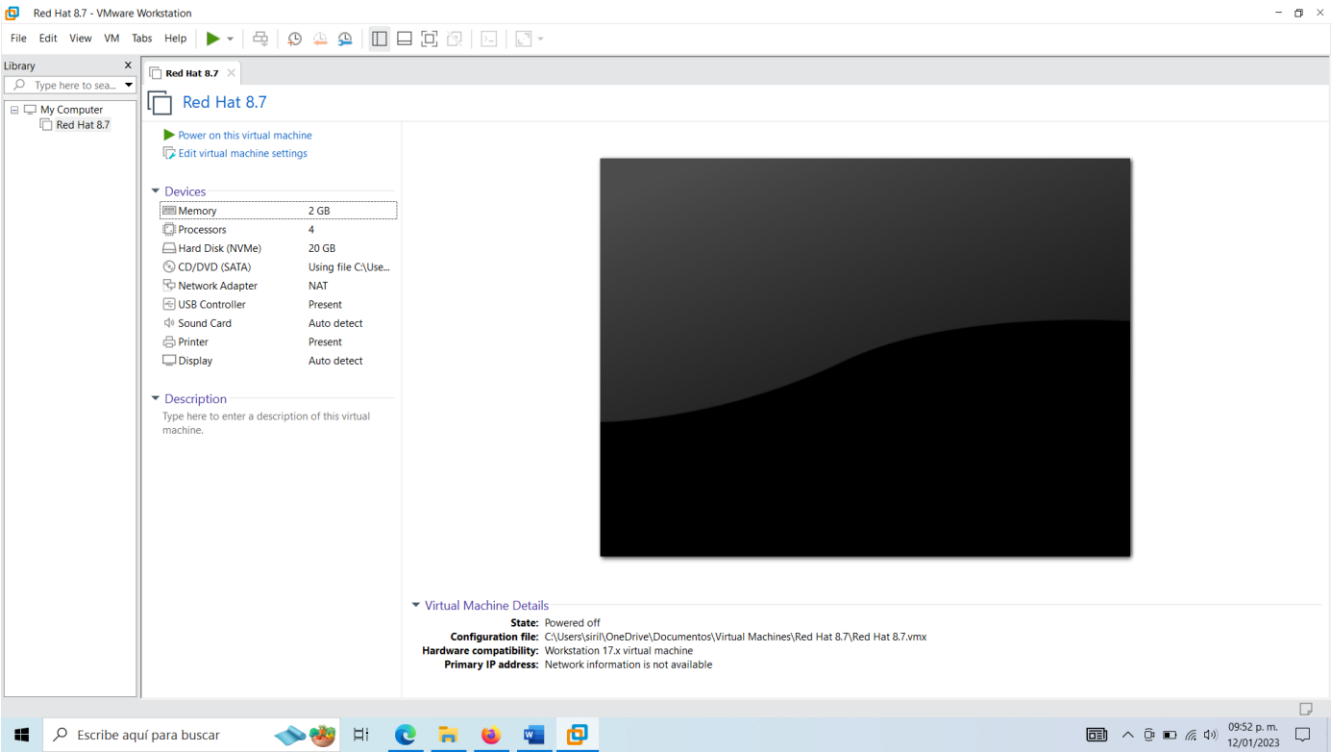
En la bibliografía del documento se plasma el enlace de referencia para realizar la instalación de este programa para Windows con una licencia activa.



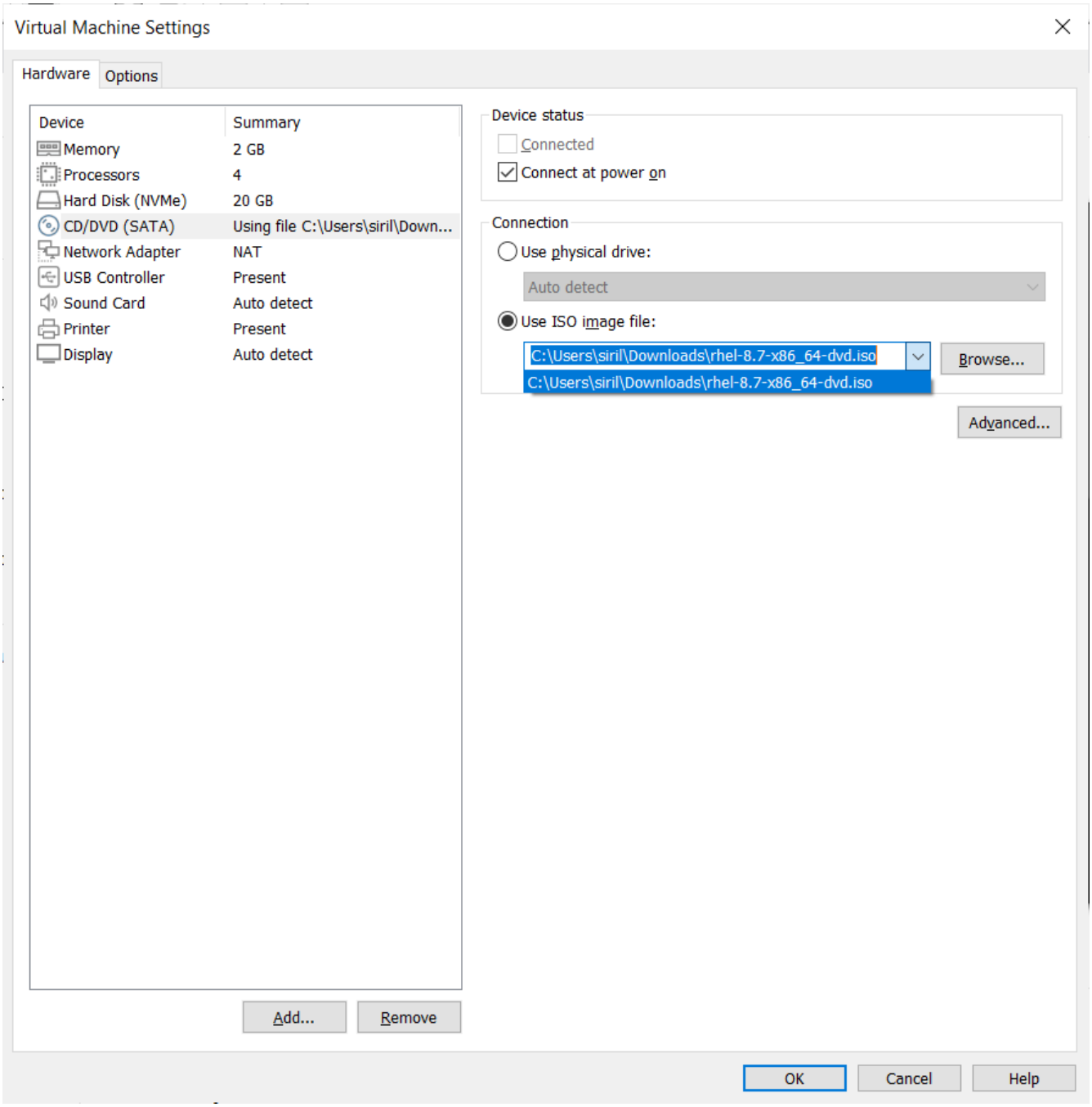
Una vez instalado MVWARE se tiene que contar con el archivo ISO del sistema que se desea instalar, en este caso se realizó una descarga e instalación de un Red Hat Enterprise Linux 8.7

Para poder descargar esta imagen es necesario registrarse en la página de Red Hat.

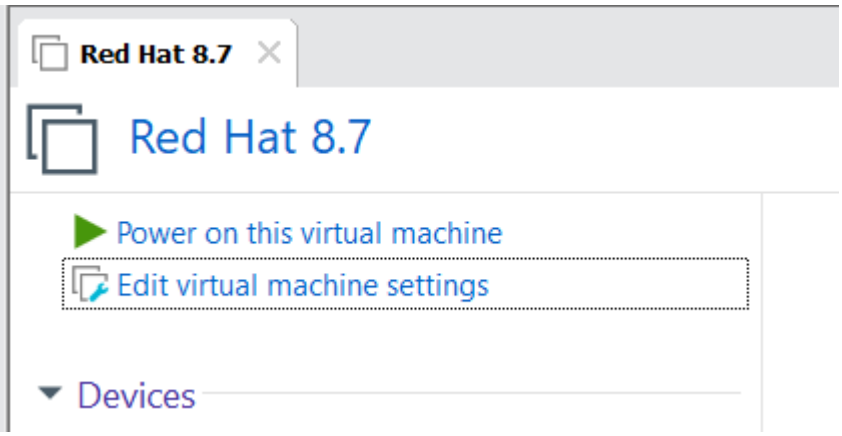
Una vez teniendo el archivo ISO, es posible proceder a crear una máquina virtual con RedHat.



Una vez creada la máquina virtual nos aparece esta ventana donde también nos muestra la configuración de la máquina. Para seleccionar el ISO es necesario editar la configuración.



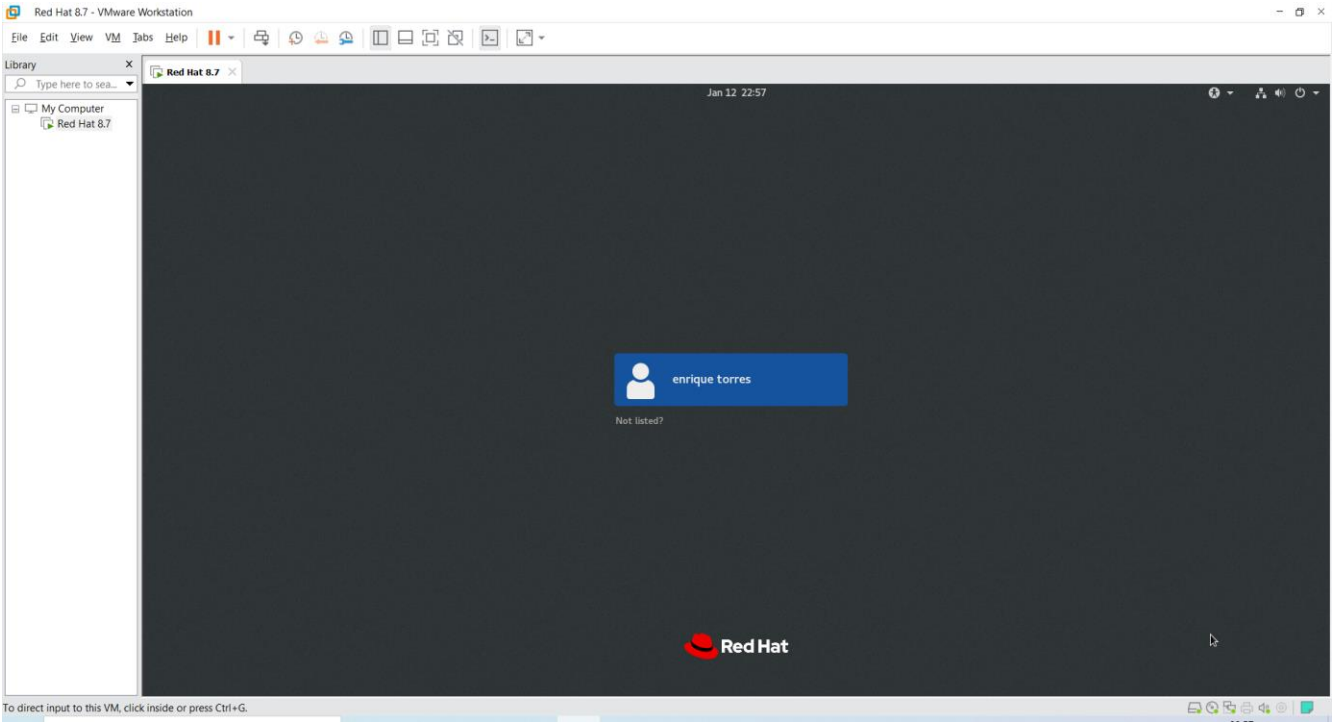
Esto lo realizamos en esta ventana, seleccionamos la opción de usar un archivo de imagen ISO, en la opción de browse… podemos seleccionar el archivo que descargamos.



Hecho todo esto podemos iniciar la máquina virtual.

Si es la primera vez que se realiza este proceso, se iniciara el instalador de Red Hat, en donde únicamente se selecciona el disco virtual que se va a usar para instalar Red Hat, es necesario crear un usuario y contraseña root, se puede configurar el idioma y teclado, para este caso se seleccionaron el idioma inglés, y teclado latinoamericano.

Cuando la instalación del sistema termina, se reinicia el sistema Red Hat y justo antes de empezar, se deben aceptar los términos de licencia, se crea un usuario normal con su nombre y contraseña y finalmente se empieza a trabajar con el sistema.



También es necesario realizar un registro del sistema para que reciba actualizaciones y se pueda trabajar sin problema con los comandos de Red Hat.

Para realizar el registro se debe ingresar el siguiente comando

```
# subscription-manager register –username <USERNAME> --password <PASSWORD> --auto-attach
```

Se tiene que ingresar en modo root desde la termina, y el nombre de usuario y contraseña que se ingresan son los mismos que se usaron para crear la cuenta de Red Hat y poder descargar la imagen ISO.

Ahora una vez que estamos listos, podemos verificar que Podman está integrado en Red Hat con el siguiente comando.

```
[root@localhost sirilo]# podman version
Client:       Podman Engine
Version:      4.2.0
API Version:  4.2.0
Go Version:   go1.18.7
Built:        Wed Oct 26 15:23:47 2022
OS/Arch:      linux/amd64
[root@localhost sirilo]#
```

Con Podman versión podemos obtener la información de Podman que tenemos.

```
[root@localhost sirilo]# yum list podman*
Updating Subscription Management repositories.
Last metadata expiration check: 16:50:45 ago on Thu 12 Jan 2023 11:30:41 PM EST.
Installed Packages
podman.x86_64                                                    3:4.2.0-4.module+el8.7.0+17064+3b31f55c
podman-catatonit.x86_64                                          3:4.2.0-4.module+el8.7.0+17064+3b31f55c
podman-docker.noarch                                             3:4.2.0-4.module+el8.7.0+17064+3b31f55c
Available Packages
podman.x86_64                                                    3:4.2.0-6.module+el8.7.0+17498+a7f63b89
podman-catatonit.x86_64                                          3:4.2.0-6.module+el8.7.0+17498+a7f63b89
podman-compose.noarch                                            1.0.3-3.el8
podman-docker.noarch                                             3:4.2.0-6.module+el8.7.0+17498+a7f63b89
podman-gvproxy.x86_64                                            3:4.2.0-6.module+el8.7.0+17498+a7f63b89
podman-manpages.noarch                                           1.6.4-4.module+el8.1.1+5885+44006e55
podman-plugins.x86_64                                            3:4.2.0-6.module+el8.7.0+17498+a7f63b89
podman-remote.x86_64                                             3:4.2.0-6.module+el8.7.0+17498+a7f63b89
podman-tests.x86_64                                              3:4.2.0-6.module+el8.7.0+17498+a7f63b89
[root@localhost sirilo]#
```

Para poder ver las características de nuestro sistema también puede ser necesario instalar neofetch

Para instalar neofetch en Red Hat son los siguientes comandos:

Para instalar neofetch en Red Hat es necesario primero instalar el repositorio de EPEL:

```
$ sudo dnf install https://dl.fedoraproject.org/pub/epel/epel-release-latest-8.noarch.rpm
$ sudo dnf install epel-release
```

Una vez que esta instalado el repositorio de EPEL podemos instalar neofetch con el siguiente comando:

```
$ sudo dnf install neofetch
```

Ahora lanzamos en commando neofetch

Ahora para buscar imágenes de contenedores de Amundsen podemos ingresar el comando “podman search amundsen”

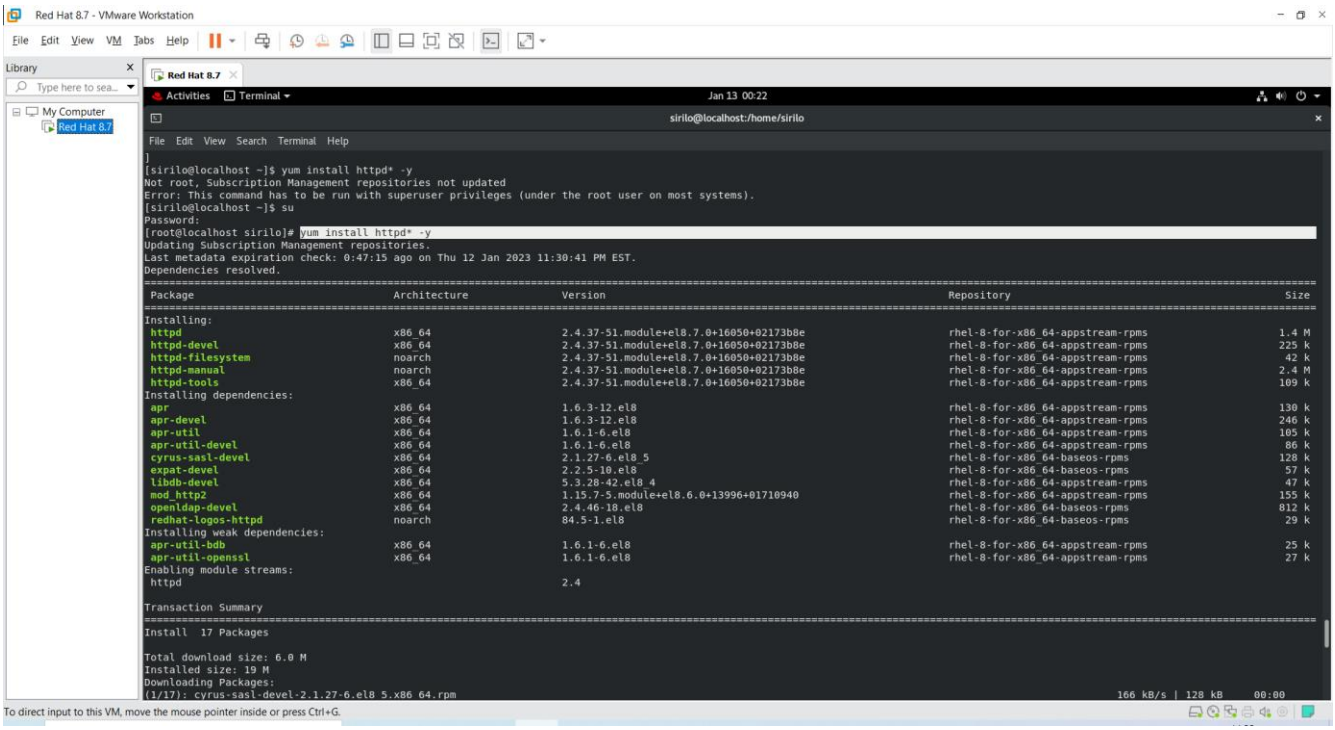
```
[root@localhost sirilo]# podman search amundsen
NAME                                                DESCRIPTION
docker.io/amundsendev/amundsen-search
docker.io/amundsendev/amundsen-metadata
docker.io/amundsendev/amundsen-frontend
docker.io/amundsendev/amundsen-frontend-oidc
docker.io/amundsendev/amundsen-metadata-oidc
docker.io/ghvadepu/amundsen-frontend-oidc
docker.io/vitorhmendes97/amundsen-search-prd
docker.io/kylg/amundsen-frontend-oidc
docker.io/aykcandem/amundsen                        amundsen data catalog
docker.io/muhraflesh/amundsenfrontend
docker.io/kylg/amundsen-preview
docker.io/kylg/amundsen-metadata-oidc
docker.io/thebramelfrink/amundsen-frontend
docker.io/tanmay76/amundsen-frontend-oidc
docker.io/joaosilvafrete/amundsen                   Imagem do amundsen buildada no dockerhub
docker.io/ant1on1su/amundsenfrontend
docker.io/ant1on1su/amundsenmetadata
docker.io/ghvadepu/amundsen-metadata
docker.io/razorpay/amundsen-frontend
docker.io/caisho/amundsen-extract-csv-metadata
docker.io/kylg/amundsen-frontend
docker.io/kylg/amundsen-metadata
docker.io/wiltonsobreira/amundsen-atlas-frontend
docker.io/cdquintas/amundsenfrontend
docker.io/tanmay76/amundsen-frontend-test           amundsen local build
[root@localhost sirilo]#
```

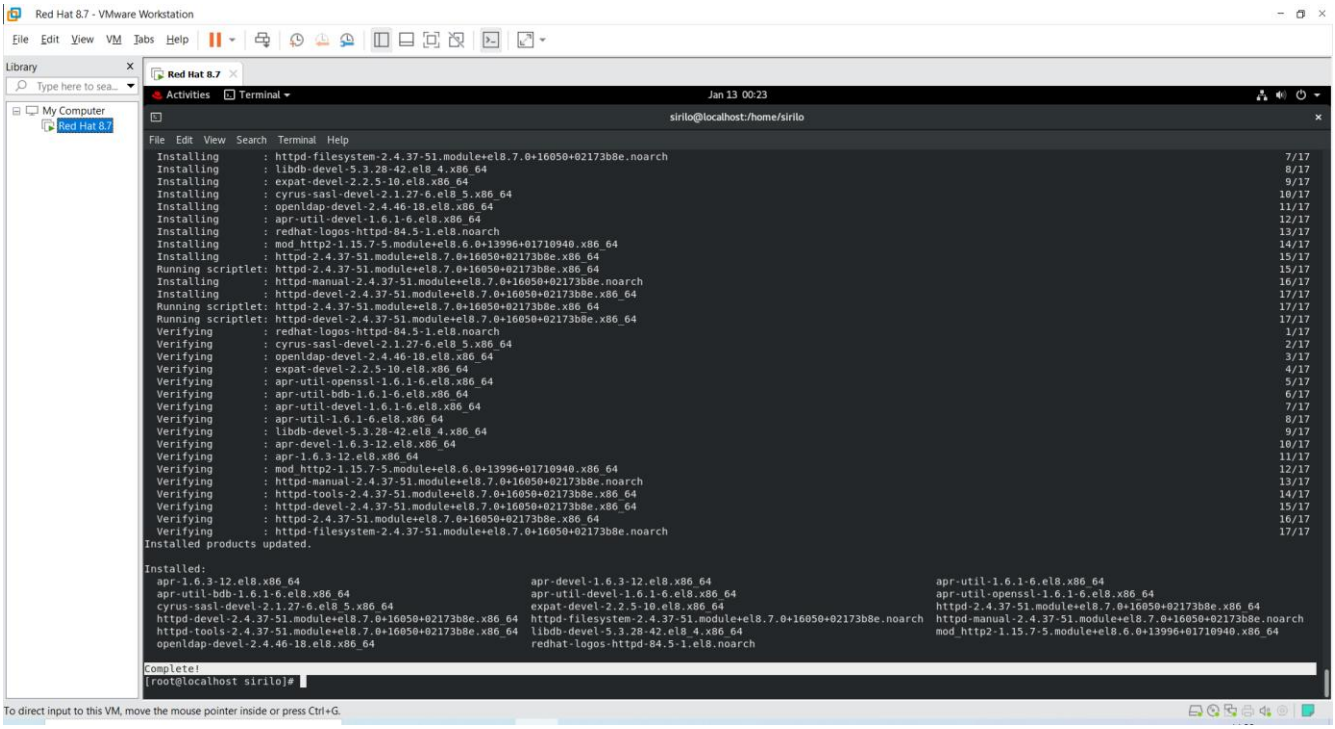
Por ejemplo, para ver las imágenes que existen de Amundsen podemos ingresar el comando Podman search Amundsen y como se ve en la imagen se despliegan todos los repositorios disponibles.

Pero antes de iniciar con Amundsen es necesario instalar y configurar el servidor http en Red Hat

Para instalarlo, tenemos que ingresar como super usuario en la terminal e ingresar el comando “yum install httpd* -y”

Con esto se instala este servidor en Red Hat.





lcid@inegi.org.mx

Ahora tenemos que ver el estado de este servicio con “service httpd status”

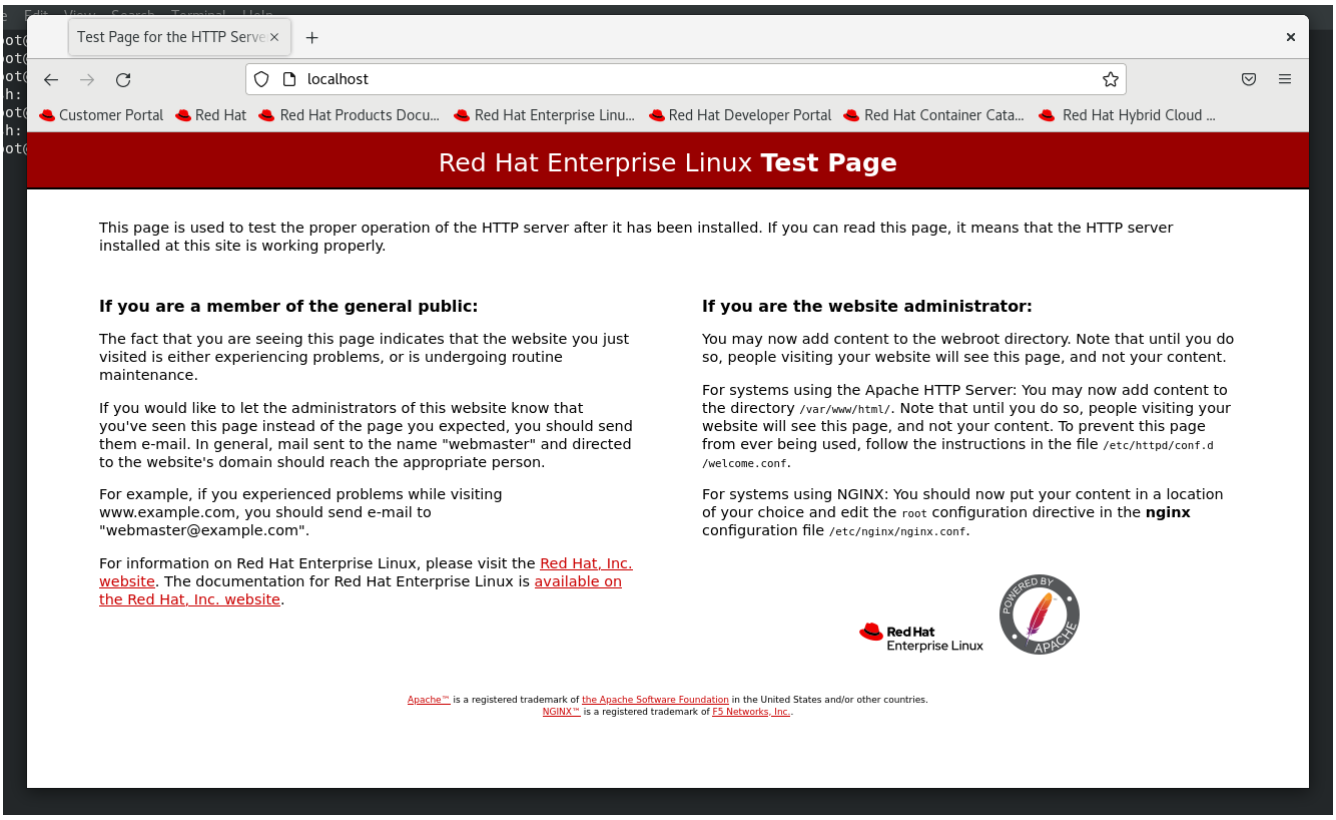
```
[root@localhost sirilo]# service httpd status
Redirecting to /bin/systemctl status httpd.service
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; vendor preset: disabled)
   Active: inactive (dead)
     Docs: man:httpd.service(8)
[root@localhost sirilo]#
```

Como vemos esta deshabilitado y para habilitar este servicio usamos el comando “service httpd start”

```
[root@localhost sirilo]# service httpd start
Redirecting to /bin/systemctl start httpd.service
[root@localhost sirilo]# service httpd status
Redirecting to /bin/systemctl status httpd.service
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; vendor preset: disabled)
   Active: active (running) since Fri 2023-01-13 00:38:52 EST; 9s ago
     Docs: man:httpd.service(8)
 Main PID: 7992 (httpd)
   Status: "Started, listening on: port 80"
    Tasks: 213 (limit: 11046)
   Memory: 18.5M
   CGroup: /system.slice/httpd.service
           ├─7992 /usr/sbin/httpd -DFOREGROUND
           ├─7999 /usr/sbin/httpd -DFOREGROUND
           ├─8000 /usr/sbin/httpd -DFOREGROUND
           ├─8042 /usr/sbin/httpd -DFOREGROUND
           └─8092 /usr/sbin/httpd -DFOREGROUND

Jan 13 00:38:52 localhost.localdomain systemd[1]: Starting The Apache HTTP Server...
Jan 13 00:38:52 localhost.localdomain httpd[7992]: AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using localhost.localdomain. Set the 'ServerName'
Jan 13 00:38:52 localhost.localdomain systemd[1]: Started The Apache HTTP Server.
Jan 13 00:38:52 localhost.localdomain httpd[7992]: Server configured, listening on: port 80
lines 1-19/19 (END)
```

Ahora lo tenemos activo, por último, se realiza un test del servicio http, para eso abrimos el navegador e ingresamos http://localhost/ en esta instancia nos debe abrir la página de Red Hat Test.



Trabajando contenedor Amundsen.

Los siguientes comandos son utilizados para trabajar un contenedor de Amundsen.

`Podman search Amundsen`

con este comando podemos buscar todos los contenedores disponibles con el nombre especificado, con este comando pues hacemos una búsqueda de los contenedores disponibles que hay de Amundsen.

Para saber cual de los contenedores del listado debemos descargar nos fijamos en la puntuación que tiene cada contenedor.

```
[root@localhost sirilo]# podman search amundsen
NAME                                          DESCRIPTION
docker.io/amundsendev/amundsen-search
docker.io/amundsendev/amundsen-metadata
docker.io/amundsendev/amundsen-frontend
docker.io/amundsendev/amundsen-frontend-oidc
docker.io/amundsendev/amundsen-metadata-oidc
docker.io/ghvadepu/amundsen-frontend-oidc
docker.io/vitorhmendes97/amundsen-search-prd
docker.io/kylg/amundsen-frontend-oidc
docker.io/aykcandem/amundsen                  amundsen data catalog
docker.io/muhraflesh/amundsenfrontend
docker.io/kylg/amundsen-preview
docker.io/kylg/amundsen-metadata-oidc
docker.io/thebramelfrink/amundsen-frontend
docker.io/tanmay76/amundsen-frontend-oidc
docker.io/joaosilvafrete/amundsen             Imagem do amundsen buildada no dockerhub
docker.io/ant1on1su/amundsenfrontend
docker.io/ant1on1su/amundsenmetadata
docker.io/ghvadepu/amundsen-metadata
docker.io/razorpay/amundsen-frontend
docker.io/caisho/amundsen-extract-csv-metadata
docker.io/kylg/amundsen-frontend
docker.io/kylg/amundsen-metadata
docker.io/wiltonsobreira/amundsen-atlas-frontend
docker.io/cdquintas/amundsenfrontend
docker.io/tanmay76/amundsen-frontend-test     amundsen local build
[root@localhost sirilo]#
```

Una vez seleccionado el contenedor, se descarga utilizando el siguiente comando

```
Podman pull docker.io/amundsendev/amundsen-frontend
```

```
[root@localhost sirilo]# podman pull docker.io/amundsendev/amundsen-frontend
Trying to pull docker.io/amundsendev/amundsen-frontend:latest...
Getting image source signatures
Copying blob 31078b2538a2 done
Copying blob f63e77b7563a done
Copying blob 42c077c10790 done
Copying blob e69ebd661d90 done
Copying blob dca49bd08fde done
Copying blob 51a05345c44d done
Copying blob 3b6aeef29d24 done
Copying blob 52af7a8c1c65 done
Copying blob 8f2af027defb done
Copying blob d518e496c9c4 done
Copying blob 624feb6228ab done
Copying blob 67191f138b1f done
Copying config 88ea40995e done
Writing manifest to image destination
Storing signatures
88ea40995e31ca78511a2c3019c1c174d31146467351c86b41be99993fc3d1a4
[root@localhost sirilo]#
```

Cuando finaliza la descarga podemos ver la imagen con

podman image list

```
[root@localhost sirilo]# podman image list
REPOSITORY                                TAG         IMAGE ID      CREATED       SIZE
quay.io/podman/hello                      latest      4d77318aea66  4 hours ago   82.6 kB
docker.io/library/ubuntu                  latest      6b7dfa7e8fdb  5 weeks ago   80.3 MB
docker.io/amundsendev/amundsen-frontend   latest      88ea40995e31  6 months ago  960 MB
quay.io/centos/centos                     latest      300e315adb2f  2 years ago   217 MB
[root@localhost sirilo]#
```

Como se puede observar se tienen actualmente algunas imágenes descargadas, pero comprobamos que ya tenemos la de Amundsen, que muestra que ah sido creada hace 6 meses aproximadamente.

El siguiente paso es correr el contenedor, para esta practica se realizó de la siguiente manera ingresando el siguiente comando.

podman run -d -p 8080:80 --name contenedor_amundsen_sirilo docker.io/amundsendev/amundsen-frontend

```
[root@localhost sirilo]# podman run -d -p 8080:80 --name contenedor_amundsen_sirilo docker.io/amundsendev/amundsen-frontend
9ce99cc7b387d787fb801df470db17b773f202a4ec0482e6b0b7423b0dd4f79b
[root@localhost sirilo]#
```

De este modo podemos ver que por el resultado que nos arroja, el contenedor ya esta corriendo y para comprobarlo lo hacemos con el siguiente comando:

podman ps

```
[root@localhost sirilo]# podman ps
CONTAINER ID  IMAGE                                       COMMAND               CREATED        STATUS            PORTS                   NAMES
9ce99cc7b387  docker.io/amundsendev/amundsen-frontend:latest  python3 amundsen_...  4 minutes ago  Up 4 minutes ago  0.0.0.0:8080->80/tcp  contenedor_amundsen_sirilo
[root@localhost sirilo]#
```

Para ver el puerto por el cual esta corriendo utilizamos el siguiente comando:

netstat -an | grep 8080

```
[root@localhost sirilo]# netstat -an | grep 8080
tcp        0      0 0.0.0.0:8080            0.0.0.0:*               LISTEN
[root@localhost sirilo]#
```

Podemos ingresar el comando “podman container –help” para ver los comandos de contenedores que se pueden realizar con podman container

```
Description:
  Manage containers

Usage:
  podman container [command]

Available Commands:
  attach      Attach to a running container
  checkpoint  Checkpoints one or more containers
  cleanup     Clean up network and mountpoints of one or more containers
  clone       Clone an existing container
  commit      Create new image based on the changed container
  cp          Copy files/folders between a container and the local filesystem
  create      Create but do not start a container
  diff        Inspect changes to the container's file systems
  exec        Run a process in a running container
  exists      Check if a container exists in local storage
  export      Export container's filesystem contents as a tar archive
  init        Initialize one or more containers
  inspect     Display the configuration of a container
  kill        Kill one or more running containers with a specific signal
  list        List containers
  logs        Fetch the logs of one or more containers
  mount       Mount a working container's root filesystem
  pause       Pause all the processes in one or more containers
  port        List port mappings or a specific mapping for the container
  prune       Remove all non running containers
  ps          List containers
  rename      Rename an existing container
  restart     Restart one or more containers
  restore     Restores one or more containers from a checkpoint
  rm          Remove one or more containers
  run         Run a command in a new container
  runlabel    Execute the command described by an image label
  start       Start one or more containers
  stats       Display a live stream of container resource usage statistics
  stop        Stop one or more containers
  top         Display the running processes of a container
  unmount     Unmounts working container's root filesystem
  unpause     Unpause the processes in one or more containers
  wait        Block on one or more containers

[root@localhost sirilo]#
```

Podemos hacer inspeccionar el contenedor de Amundsen para ver la información que tiene con “podman container inspect contenedor_amundsen_sirilo”

```
[root@localhost sirilo]# podman container inspect contenedor_amundsen_sirilo
[
    {
        "Id": "472fb8e6d7fe0abb9b034f983fd761d6ec0eead9a3d3200a2578fbb5a2852065",
        "Created": "2023-01-13T17:50:28.630445128-05:00",
        "Path": "python3",
        "Args": [
            "amundsen_application/wsgi.py"
        ],
        "State": {
            "OciVersion": "1.0.2-dev",
            "Status": "running",
            "Running": true,
            "Paused": false,
            "Restarting": false,
            "OOMKilled": false,
            "Dead": false,
            "Pid": 8742,
            "ConmonPid": 8723,
            "ExitCode": 0,
            "Error": "",
            "StartedAt": "2023-01-13T17:50:30.325265822-05:00",
            "FinishedAt": "0001-01-01T00:00:00Z",
            "Health": {
                "Status": "",
                "FailingStreak": 0,
                "Log": null
            },
            "CgroupPath": "/machine.slice/libpod-472fb8e6d7fe0abb9b034f983fd761d6ec0eead9a3d3200a2578fbb5a2852065.scope",
            "CheckpointedAt": "0001-01-01T00:00:00Z",
            "RestoredAt": "0001-01-01T00:00:00Z"
        },
        "Image": "88ea40995e31ca78511a2c3019c1c174d31146467351c86b41be99993fc3d1a4",
        "ImageName": "docker.io/amundsendev/amundsen-frontend:latest",
        "Rootfs": "",
        "Pod": "",
        "ResolvConfPath": "/run/containers/storage/overlay-containers/472fb8e6d7fe0abb9b034f983fd761d6ec0eead9a3d3200a2578fbb5a2852065/userdata/resolv.conf",
        "HostnamePath": "/run/containers/storage/overlay-containers/472fb8e6d7fe0abb9b034f983fd761d6ec0eead9a3d3200a2578fbb5a2852065/userdata/hostname",
        "HostsPath": "/run/containers/storage/overlay-containers/472fb8e6d7fe0abb9b034f983fd761d6ec0eead9a3d3200a2578fbb5a2852065/userdata/hosts",
        "StaticDir": "/var/lib/containers/storage/overlay-containers/472fb8e6d7fe0abb9b034f983fd761d6ec0eead9a3d3200a2578fbb5a2852065/userdata",
        "OCIConfigPath": "/var/lib/containers/storage/overlay-containers/472fb8e6d7fe0abb9b034f983fd761d6ec0eead9a3d3200a2578fbb5a2852065/userdata/config.json",
ove the mouse pointer outside or press Ctrl+Alt.
```

Podemos ver que nos despliega bastante información sobre el contenedor, donde podemos encontrar información interesante.

También con el comando “podman start –all” podemos levantar todos los contenedores que tengamos disponibles

```
[root@localhost sirilo]# podman start --all
249761021f0dd306709f6233405c76b78cd07581ab28e0269a4321947f3f8c9e
421ac4b824e32346cd82aedffa03459e3f37623fca98943c9f4956a39d3c5c90
76c85e7168ffa7ca2ec94dc6d1f3cfe12aeccd9ee8ffeaad1015fbf11df85aba
bd5e9a3502cb3af5d6230d3ae6108a5227b75ade0eb6a5f7c57853376a9b7004
[root@localhost sirilo]# podman ps
CONTAINER ID  IMAGE                                       COMMAND               CREATED        STATUS            PORTS                   NAMES
bd5e9a3502cb  docker.io/library/ubuntu:latest            /bin/bash             2 days ago     Up 5 seconds ago                          siriloContainer
472fb8e6d7fe  docker.io/amundsendev/amundsen-frontend:latest  python3 amundsen_...  7 minutes ago  Up 7 minutes ago  0.0.0.0:8081->80/tcp  contenedor_amundsen_sirilo
[root@localhost sirilo]#
```

Y con el comando “podman stop –all” podemos bajar o detener todos los contenedores activos.

```
[root@localhost sirilo]# podman stop --all
WARN[0010] StopSignal SIGTERM failed to stop container siriloContainer in 10 seconds, resorting to SIGKILL
WARN[0010] StopSignal SIGTERM failed to stop container contenedor_amundsen_sirilo in 10 seconds, resorting to SIGKILL
472fb8e6d7fe0abb9b034f983fd761d6ec0eead9a3d3200a2578fbb5a2852065
76c85e7168ffa7ca2ec94dc6d1f3cfe12aeccd9ee8ffeaad1015fbf11df85aba
bd5e9a3502cb3af5d6230d3ae6108a5227b75ade0eb6a5f7c57853376a9b7004
249761021f0dd306709f6233405c76b78cd07581ab28e0269a4321947f3f8c9e
421ac4b824e32346cd82aedffa03459e3f37623fca98943c9f4956a39d3c5c90
[root@localhost sirilo]# podman ps
CONTAINER ID  IMAGE       COMMAND     CREATED     STATUS      PORTS       NAMES
[root@localhost sirilo]#
```

Bibliografía.

[1] https://todobi.com/apache-atlas-para-tus-procesos-de-datos/

[2] https://www.amundsen.io/amundsen/metadata/

[3] https://businessyield.com/es/management/metadata-management-tools/

[4] https://atlan.com/atlas-setup-tutorial/

[5] https://www.youtube.com/watch?v=BuCgnhCVUqw

[6] https://www.amundsen.io/amundsen/metadata/

[7] https://www.amundsen.io/amundsen/

[8] https://www.youtube.com/watch?v=2OTZZlufj-A&list=PLkjz14EiPnOB_Zgr_hHlriHNY3JpBcW0B&index=1

[9] https://www.youtube.com/watch?v=RD2vqDsp4hQ&list=PLkjz14EiPnOB_Zgr_hHlriHNY3JpBcW0B&index=2

[10] https://www.youtube.com/watch?v=QmfkZH7LVtI&list=PLkjz14EiPnOB_Zgr_hHlriHNY3JpBcW0B&index=4

[11] https://linuxways.net/centos/how-to-install-and-use-neofetch-on-linux-rhel-centos-arch/

[12] https://www.youtube.com/watch?v=CdXjI81py5o&list=PLkjz14EiPnOB_Zgr_hHlriHNY3JpBcW0B&index=5

[13] https://www.youtube.com/watch?v=uay5RhFrbNQ

[14] https://youtu.be/uay5RhFrbNQ